WebSerial Vision Training for Microcontrollers: A Browser-Based Companion to On-Device CNN Training

webmcu-vision-web (Paper 2 of the webmcu-ai Series)

Version 1.0.0

Jeremy Ellis
High School Robotics Educator
British Columbia, Canada
https://github.com/webmcu-ai

April 2026

Abstract

This paper presents `webmcu-vision-web`, a single-file, zero-install browser application for end-to-end TinyML vision model training and deployment on the Seeed Studio XIAO ESP32-S3 Sense (XIAO ML Kit, $15–40 USD). Acting as a browser-based companion to the on-device Arduino firmware of Paper 1 [1], it provides a private, fully local machine learning pipeline—from firmware flashing through image collection, CNN training, weight export, and live activation visualization—without any software installation beyond a Chromium-based browser. The system targets educators, small businesses, and researchers who need to train task-specific visual classifiers under their exact deployment conditions rather than adapting large general-purpose datasets.

Key capabilities include: in-browser firmware flashing via `esptool-js`; a bidirectional serial monitor; a full SD card file browser with image preview and inline editing; `config.json` live-sync for zero-recompile hyperparameter adjustment; webcam and ESP32 OV2640 camera image capture; TensorFlow.js CNN training matching the on-device architecture, completing a representative three-class run (∼30 images per class, 20 epochs) in approximately 1 minute browser-side versus 9 minutes on-device, enabling a complete collect–train–deploy cycle in under 10 minutes; weight export as `myWeights.bin` and `myWeights.h` for deployment to the SD card; confusion matrix; and a live Conv2 activation heatmap streamed from the ESP32 during inference. No data leaves the local machine at any stage. A five-run consistency evaluation on the three-class reference problem (0Blank, 1Cup, 2Pen) with independently collected datasets demonstrates stable convergence and provides mean accuracy with standard deviation; all artefacts and raw results are released at the repository link below. The repository is structured as a living template designed for LLM-assisted adaptation to new hardware, sensors, and tasks. All source code is released under the MIT License at https://github.com/webmcu-ai/webmcu-vision-web.

Keywords: WebSerial, TinyML, browser-based training, TensorFlow.js, ESP32-S3, embedded machine learning, edge AI, engineering education, firmware flashing, on-device vision, LLM-assisted development.

Contents

1 Introduction **3**
- 1.1 The TinyML Dataset Philosophy . . . 3
- 1.2 Problem Statement . . . 4
- 1.3 LLM-Assisted Adaptation . . . 4
- 1.4 Contributions . . . 4

2 Related Work **5**

3 System Overview **6**
- 3.1 Relationship to Paper 1 . . . 6
- 3.2 Architecture and Single-File Design . . . 6
- 3.3 Hardware and Software Requirements . . . 6

4 Application Sections **10**
- 4.1 In-Browser Firmware Flashing . . . 10
- 4.2 WebSerial Monitor and SD Card Browser . . . 11
  - 4.2.1 Live `config.json` Sync . . . 11
- 4.3 Camera and Data Collection . . . 11
- 4.4 Browser-Side CNN Training . . . 13
- 4.5 Weight Export to SD Card . . . 15
  - 4.5.1 `myWeights.bin` . . . 15
  - 4.5.2 `myWeights.h` . . . 16

5 Conv2 Activation Heatmap **16**

6 Experimental Evaluation **19**
- 6.1 Setup . . . 19
- 6.2 Consistency Evaluation: Five Independent Runs . . . 19
- 6.3 Training Accuracy vs. Real-World Inference . . . 19
- 6.4 Training Speed and Accuracy . . . 22
- 6.5 Dataset Size and Sufficiency . . . 24

7 WebSerial Communication Protocol **25**

8 Comparison to Paper 1 Workflow **25**

9 Limitations **26**

10 Repository and Reproducibility **27**

11 Future Work **28**

12 Conclusion **29**

1 Introduction

Paper 1 of this series [1] demonstrated that a complete CNN training pipeline—data capture, backpropagation, Adam optimization [2], and 6.3 FPS inference—executes entirely on a $15–40 USD microcontroller with no external computation, completing one training run (∼90 images, 20 epochs) in approximately 9 minutes. That paper defines Tier 1 of a three-tier mental model for embedded ML education (Table 1).

Table 1: Three-tier ML deployment model (from Paper 1 [1]). Tier 2 occupies the pedagogical sweet spot: it retains the local-data privacy guarantee of Tier 1 while providing the UI richness and training speed of a laptop.

Tier	**Infrastructure**	**When to choose**	**Privacy**
1. On-device	Microcontroller only	Sensitive data, no connectivity, ultra-low power, transparent education	Raw data stays on-device
2. Local computer	Laptop + USB (WebSerial)	Larger datasets, classroom use, faster iteration, richer UI	Data stays on local machine
3. Cloud	Remote GPU cluster	Production scale, large teams, foundation models	Data leaves premises

1.1 The TinyML Dataset Philosophy

A key distinction of TinyML relative to large-scale computer vision is the role of the dataset. In cloud-based ML, model generality is a virtue: large datasets covering diverse conditions produce models that work everywhere. TinyML inverts this priority. A device consuming milliwatts must solve *one specific task*—detecting a particular part on a factory line under known fluorescent lighting, or identifying a specific plant disease in a single greenhouse. The correct dataset for this task is not a large general corpus: it is a small, clean, purposefully collected set of images taken *under exactly the conditions of deployment*.

This paper's system is designed around that philosophy. The camera is positioned in front of the actual objects. Images are captured under the actual lighting. The classes are named for the actual problem. The model is trained until it converges on that specific problem, not on a benchmark. This approach yields smaller datasets (15–100 images per class is typically sufficient), faster training, lower energy consumption at inference time, and higher accuracy on the target task than a general model ever could.

This matters especially for the global south and for small businesses, where access to cloud GPU infrastructure may be intermittent or costly, and where the specific deployment conditions differ substantially from any publicly available dataset. A worker in a materials-sorting facility, a researcher studying local crop diseases, a teacher in a school with unreliable internet: all can collect their own images, train on their own hardware, and deploy a working classifier without sending any data off-premises.

1.2 Problem Statement

Paper 1 of this series [1] establishes a fully self-contained on-device ML pipeline, but operating it entirely through the device's own interface presents practical friction points for classroom and rapid-iteration use. Firmware installation for a new device requires the Arduino IDE with the ESP32 board package installed. Reading collected images or adjusting training parameters means ejecting the SD card and editing files on a PC. Any hyperparameter change requires a source-code edit and a full firmware recompile. Capturing training images is done through an OLED-based touch menu on the device itself, with a very small black-and-white OLED image preview. Collectively, these friction points can consume the majority of a class session before any ML concept is taught.

This paper addresses each of these Paper 1 workflow limitations directly with a browser-native companion interface requiring no software installation beyond Chrome or Edge.

1.3 LLM-Assisted Adaptation

This repository is explicitly designed as a template for LLM-assisted modification. The single-file structure, consistent naming conventions, and inline documentation are all chosen to make the codebase readable and modifiable by a practitioner using Claude, ChatGPT, Gemini, or any other LLM assistant. A user in any country can describe their specific hardware variant, sensor modality, or class problem to an LLM and receive targeted diffs to the relevant sections of the file. This paper documents the architecture precisely enough that those modifications are reproducible and verifiable. The webmcu-ai GitHub organization is structured as a series of such templates, each covering a different sensor modality (vision, audio, IMU), with the goal that any researcher or small-business practitioner can adapt them with minimal barrier.

1.4 Contributions

This paper makes four primary contributions:

1. **A fully local, zero-install ML lifecycle for embedded systems.** The complete TinyML workflow—firmware flash, image collection, CNN training, weight transfer, and activation visualization—executes from a single HTML file with no installed toolchain, no cloud dependency, and no data leaving the local machine. This is, to our knowledge, the first such end-to-end system for microcontroller-class hardware delivered entirely in a browser.

2. **A new pedagogical model for TinyML iteration.** Separating the fast browser loop ($\sim$1 min per run, WebGL-accelerated) from the slow embedded loop ($\sim$9 min per run, ESP32 FPU) creates a two-speed development model: practitioners iterate on dataset and hyperparameters at browser speed, then verify and deploy at device speed. A complete collect–train–deploy cycle takes under 10 minutes, supporting multiple full iterations within a single class period.

3. **A deployment philosophy for intentional, task-specific overfitting.** TinyML classifiers are not intended to generalise beyond their deployment environment; they are designed to overfit to a specific object, under specific lighting, at a specific position. This is a feature, not a limitation. The data collection workflow, camera positioning, and evaluation methodology in this system are built around this principle, making it directly applicable to global south and small-business contexts where cloud-scale datasets do not match local conditions.

4. **Live on-device activation heatmap streaming.** A Conv2 heatmap protocol streams

spatial activation maps from the running ESP32 to the browser in real-time, making the network's attention visible to the practitioner during inference. To our knowledge, this is one of the first real-time activation visualizations deployed on a microcontroller-class device. A complementary browser-side heatmap computed from the TensorFlow.js model is also available without device connection.

Supporting capabilities enabling these contributions include: SD card reading over serial; `config.json` live-sync for zero-recompile hyperparameter adjustment; exact weight format transpositions between TensorFlow.js and ESP32 C++ loop order; and a single-file repository structure designed for LLM-assisted adaptation to new hardware and sensor modalities.

2 Related Work

Edge Impulse [3] provides a professional cloud-based pipeline for TinyML: data management, training, and model export to a wide range of embedded targets. It requires account creation, uploads data to a remote server, and does not support on-device training or fully local operation. It is the correct choice for production-grade embedded ML at scale; the present work targets the complementary use case of zero-install, fully local operation where data privacy and offline access are priorities.

Google Teachable Machine [4] enables browser-based image classification training using TensorFlow.js with a minimal drag-and-drop interface. It does not support WebSerial communication, embedded firmware deployment, SD card management, or on-device inference. Its target audience is non-technical users rather than engineering students or practitioners exploring the full ML system stack.

Arduino IDE [5] and TensorFlow Lite Micro [6]. The standard embedded ML workflow for microcontrollers typically involves training a model in Python, converting it to a C array using TensorFlow Lite, and embedding that array in a C++ firmware sketch for deployment. This is a well-supported path and microcontrollers can also run Python via MicroPython or CircuitPython, though these sacrifice the performance of native C++. ML inference can additionally be written directly in C++ without a framework. The fragmentation arises from the toolchain: every model iteration requires a separate conversion step and a full firmware recompile. The system described here reduces iteration friction by allowing browser-side training and weight export in a format the firmware loads directly at boot, without any Python conversion step.

WebSerial and WebUSB educational platforms. The Web Serial API [7] has been used for browser-based microcontroller communication in tools such as Arduino Web Editor and MicroPython's WebREPL, but these focus on code editing and serial monitoring rather than ML training or embedded weight deployment.

TinyML4D [8] scales embedded ML education globally through open curricula, emphasising the importance of the full lifecycle. The system described here provides a concrete, low-cost, zero-install artefact suitable for TinyML4D-style curricula where software installation is restricted or unreliable.

Table 2 summarises the qualitative comparison. The comparison emphasises workflow characteristics rather than absolute capability; tools like Edge Impulse provide significantly broader production support and should be the default choice for teams with cloud access and production-scale deployment requirements.

Table 2: Qualitative comparison of embedded ML development approaches.

Feature	This work	Edge Impulse	Teachable Machine	Arduino + TFLite
Zero software install	✓	✗	✓	✗
Fully local data	✓	✗	✗	✓
Browser-side training	✓	✗	✓	✗
On-device training	✓	✗	✗	✗
SD card reading	✓	✗	✗	✗
Firmware flash (.ino / browser)	✓	✓	✗	✓
Activation heatmap	✓	✗	✗	✗
Config without recompile	✓	✓	✓	✗
LLM-adaptable template	✓	✗	✗	∼
Production-grade targets	✗	✓	∼	✓

3 System Overview

3.1 Relationship to Paper 1

The on-device firmware (Paper 1) is the authoritative training engine for Tier 1 deployments. `webmcu-vision-web` is not a replacement: it is a complementary interface that enables the same workflow through a browser when a laptop with a USB connection is available. The two approaches share the same weight format (`myWeights.bin`, `myWeights.h`), and the same SD card directory layout, so weights trained in the browser are directly deployable on the device and vice versa. Figure 1 shows the overall data flow.

3.2 Architecture and Single-File Design

The application is a single HTML file (`index.html`) of approximately 4,000 lines. No framework, build step, or package manager is required. External dependencies are loaded at runtime from public CDNs:

- **TensorFlow.js 4.22.0** [9] — browser-side CNN training and inference, GPU-accelerated via WebGL, matching the two-layer on-device architecture.
- **JSZip 3.10.1** — client-side ZIP creation for bulk image export.
- **esptool-js 0.5.7** [10] — WebAssembly port of `esptool.py` for in-browser firmware flashing.

All application logic—WebSerial communication, SD browser, training loop, weight import/export, heatmap rendering—is implemented in vanilla JavaScript within the single file. The single-file constraint is deliberate: it maximises accessibility, eliminates dependency management as a barrier, and makes the complete source directly auditable in a browser's developer tools. It also makes LLM-assisted modification practical: the entire codebase fits in a single context window, and targeted changes to any section can be described precisely by reference to the section numbering and function names documented here.

3.3 Hardware and Software Requirements

All experiments were conducted using a Seeed Studio XIAO ESP32-S3 Sense board as the XIAO ML kit (8 MB PSRAM, OV2640 camera, MicroSD slot) running firmware v1.0.0 of `webmcu-vision-web`.

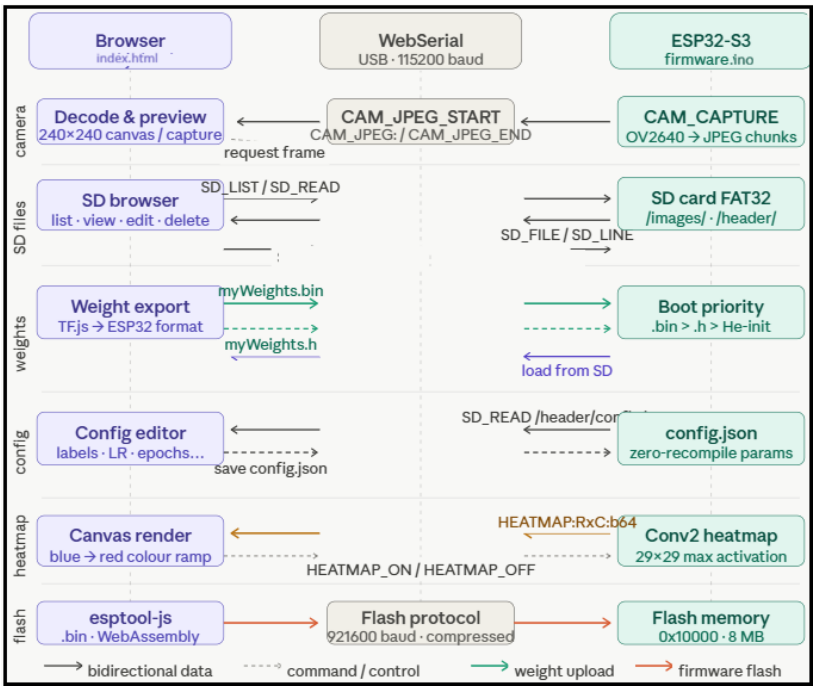

Figure 1: Data flow between `webmcu-vision-web` and the ESP32-S3 firmware. All communication passes over a single USB serial connection at 115200 baud via the browser WebSerial API. Six channels are multiplexed over this connection: camera frames (inbound), SD file operations (bidirectional), weight and config files (outbound), heatmap activation frames (inbound), and firmware flash (outbound). No data leaves the local machine.

The browser application was tested on Chrome 124 and Edge 124 (Windows 11 and macOS 14). Figure 3 shows the complete interface.

The target hardware is the Seeed Studio XIAO ESP32-S3 Sense (Figure 2). Three entry points are available at different price levels: the bare ESP32-S3 chip alone is available for approximately $8 USD; the XIAO ESP32-S3 Sense board with integrated OV2640 camera retails for approximately $15 USD; and the full XIAO ML Kit adding an OLED display, IMU, and expansion board is approximately $38.90 USD. All three configurations run the same firmware; the OLED and IMU features degrade gracefully if the expansion board is absent. For educators and students, this price range places a complete on-device ML platform within reach of a single classroom budget line.

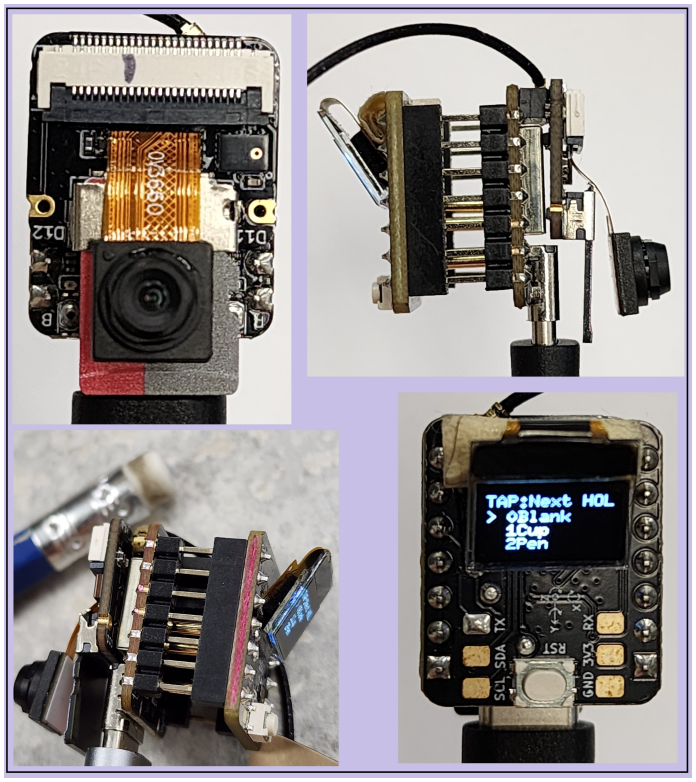

Figure 2: The XIAO ML Kit showing the ESP32-S3 Sense board with integrated OV2640 camera, 72×40 OLED display, and IMU sensor ($38.90 USD complete kit; $14.99 bare Sense board; $8 ESP32-S3 chip alone). The 8 MB PSRAM is sufficient for the full browser-assisted training pipeline. Pin A0 capacitive touch enables on-device menu navigation without the browser connection. All three hardware configurations are supported by the firmware; only the OLED and IMU features require the expansion board.

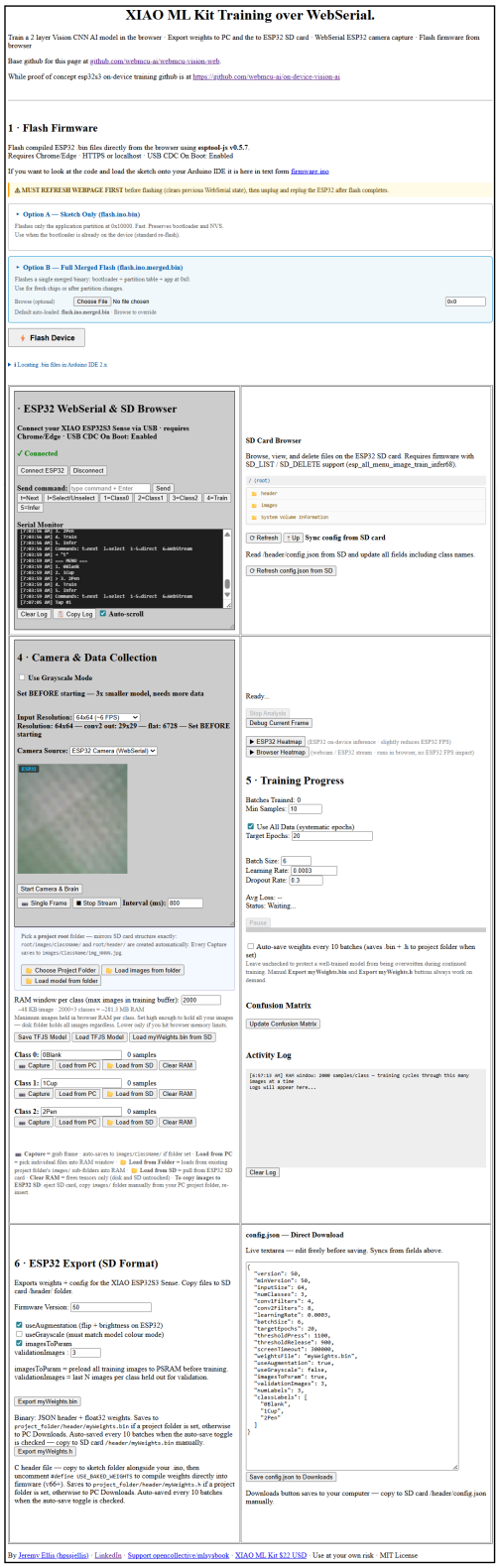

Figure 3: The complete `webmcu-vision-web` interface (`index.html`). All sections are visible on a single scrolling page with no navigation or routing. The six numbered sections (Flash Firmware, WebSerial & SD Browser, Camera & Data Collection, Training Progress, ESP32 Weight Export, and Activity Log) are self-contained and independently usable. The entire source is auditable in the browser developer tools.

4 Application Sections

4.1 In-Browser Firmware Flashing

Section 1 allows educators and practitioners to flash pre-compiled ESP32 firmware binaries directly from the browser, with no Python, `pip`, `esptool.py`, or Arduino IDE required. This eliminates the most common barrier to first-time deployment of embedded ML systems, and is particularly significant in environments where software installation is restricted by IT policy or unreliable internet access limits package downloads.

Note (v1.0.0): In-browser firmware flashing via `esptool-js` works correctly in the normal use case: the `esptool-js` library is loaded from its CDN when the browser first opens the page with an internet connection, and the flash function then operates fully locally over WebSerial even if internet is subsequently unavailable. The limitation arises only when the browser is opened on a computer that has never loaded the page before and has no internet connection at that moment, in which case the CDN library cannot be fetched. In that scenario, flashing `firmware.ino` via the Arduino IDE [5] is the fallback; all remaining pipeline stages (image collection, browser training, weight export, SD management) continue to work without internet once the page has been loaded. The portable folder in the repository includes all CDN libraries vendored locally, allowing the page except firmware flash to operate offline once downloaded. Use the Arduino IDE offline to load the `firmware.ino`.

Flashing is handled by `esptool-js` [10], a WebAssembly port of Espressif's official flashing tool. Two modes are supported:

- **Option A — Sketch only** (`.ino.bin` at `0x10000`): flashes only the application partition. Fast; preserves the bootloader and NVS storage. The correct choice for standard re-flashing once the bootloader is present.
- **Option B — Full merged binary** (`.ino.merged.bin` at `0x0`): flashes bootloader, partition table, and application in a single pass. Required for fresh chips or after partition changes.

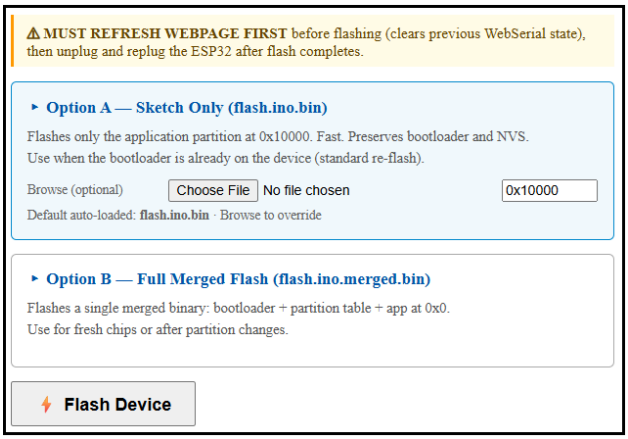

Figure 4: Section 1: In-browser firmware flashing via `esptool-js`. No Python or Arduino IDE is required. Option A (sketch-only at `0x10000`) is the default for standard re-flashing; Option B flashes a complete merged binary for fresh devices. A progress bar provides real-time feedback; the WebSerial monitor reconnects automatically after flashing.

4.2 WebSerial Monitor and SD Card Browser

Section 2 provides a two-panel interface: a live serial monitor and a graphical SD card browser, connected to the ESP32 at 115200 baud via the WebSerial API.

The serial monitor displays all firmware output in a scrollable monospace terminal with optional auto-scroll. Quick-send buttons map to the firmware's single-character command interface; a free-text command field accepts any protocol string.

The SD browser renders the full directory tree of the ESP32 SD card. Features include breadcrumb navigation, JPEG inline preview (without downloading to host), inline JSON and text editing with save-back to the device, and file delete—all without ejecting the SD card.

4.2.1 Live `config.json` Sync

A *Refresh config.json from SD* button reads `/header/config.json` and populates all corresponding browser fields: class labels, learning rate, batch size, target epochs, augmentation flag, grayscale mode, PSRAM preload flag, and validation image count. This enables zero-recompile hyperparameter adjustment: the practitioner edits `config.json` in the SD browser, saves it back, and the firmware reads the new values at the next boot without any code change. The schema is shared exactly between browser and firmware (Listing 1).

```
{
  "version": 68,
  "inputSize": 64,
  "numClasses": 3,
  "classLabels": ["0Blank", "1Cup", "2Pen"],
  "learningRate": 0.0003,
  "batchSize": 6,
  "targetEpochs": 20,
  "useAugmentation": true,
  "useGrayscale": false,
  "imagesToPsram": true,
  "validationImages": 3,
  "weightsFile": "myWeights.bin"
}
```

Listing 1: `config.json` schema shared between browser and firmware. Reading from SD populates all browser training fields; saving back takes effect at the next device boot with no recompile. The `classLabels` array drives both the ESP32 menu and the browser capture rows, making it the single source of truth for class naming.

4.3 Camera and Data Collection

Section 3 is the data collection pipeline, and its design reflects the TinyML dataset philosophy described in Section 1.1: images should be collected under the actual conditions of deployment, not adapted from a general dataset. The practitioner positions the camera in front of the real objects, under the real lighting, and captures images that the model will actually encounter at inference time.

Two camera sources are supported:

- **Webcam**: any browser-accessible camera via `getUserMedia()`, with device selection from a populated dropdown.

config.json — Direct Download

Live textarea — edit freely before saving. Syncs from fields above.


```
{
  "version": 50,
  "minVersion": 50,
  "inputSize": 64,
  "numClasses": 3,
  "conv1Filters": 4,
  "conv2Filters": 8,
  "learningRate": 0.0003,
  "batchSize": 6,
  "targetEpochs": 20,
  "thresholdPress": 1100,
  "thresholdRelease": 900,
  "screenTimeout": 300000,
  "weightsFile": "myWeights.bin",
  "useAugmentation": true,
  "useGrayscale": false,
  "imagesToPsram": true,
  "validationImages": 3,
  "numLabels": 3,
  "classLabels": [
    "0Blank",
    "1Cup",
    "2Pen"
  ]
}
```


Save config.json to Downloads

Downloads button saves to your computer — copy to SD card /header/config.json manually.

Figure 5: The `config.json` textarea in Section 5, kept live in sync with all training fields. The `config.json` file can be edited in the SD card browser (Section 2) and saved back to the device, or downloaded to the PC project folder and copied to the microSD card manually. The same JSON is read by the ESP32 firmware at boot, making it the single shared configuration source between the browser and the device.

- **ESP32 camera (WebSerial)**: live JPEG frames from the on-device OV2640 via the `CAM_CAPTURE` serial command, arriving as base64-encoded chunks and rendered in the browser at a configurable interval.

An optional grayscale mode and a resolution selector (64–240 px, with FPS estimates) must be configured before starting the training pipeline. A live hint displays the Conv2 output size and flattened dimension for the selected resolution, making the relationship between input size and model capacity explicit.

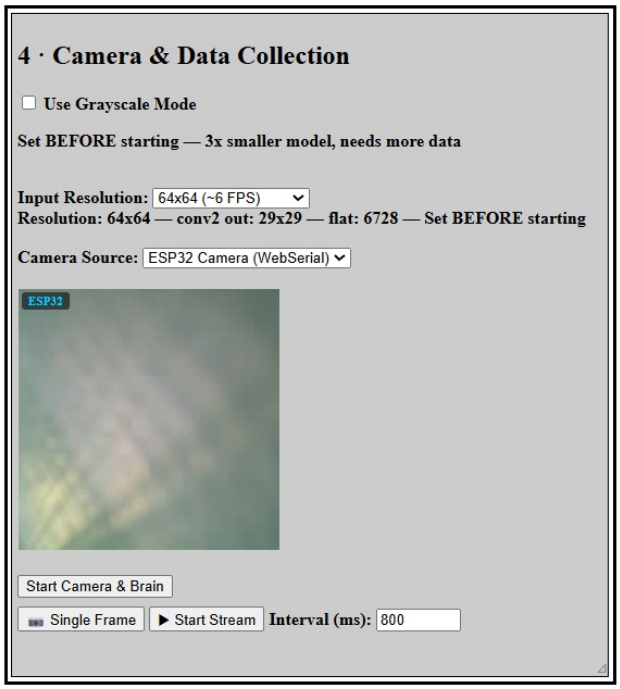

Figure 6: Section 3: Camera data collection. The practitioner captures images of their actual objects under actual deployment lighting conditions. For a fast iteration cycle, ∼30 images per class is sufficient to complete training in approximately 1 minute. Images are saved automatically to a structured PC project folder; the entire folder is copied to the microSD card as a single deployment step. The live preview and per-class sample counters give immediate feedback on dataset balance.

4.4 Browser-Side CNN Training

Section 4 runs a CNN in the browser using TensorFlow.js [9], GPU-accelerated via WebGL. The architecture exactly mirrors the on-device network (Table 3).

The training loop runs on a 150 ms `setInterval`, interleaved with live inference so the display remains responsive. Each step assembles a mini-batch (default size 6), optionally applies stochastic augmentation (brightness jitter and contrast scaling matching the firmware's augmentation pass), calls `model.trainOnBatch()`, and accumulates loss. Every 10 batches the batch count, average loss, and a qualitative status label (*Improving*, *Converging*, *Well Trained*) are updated.

Two epoch modes are selectable at runtime: *Use All Data* (systematic epoch ordering) and *random batch mode* (uniform sampling for exploratory training). Pause and Resume buttons interrupt and restart training without discarding weights or data.

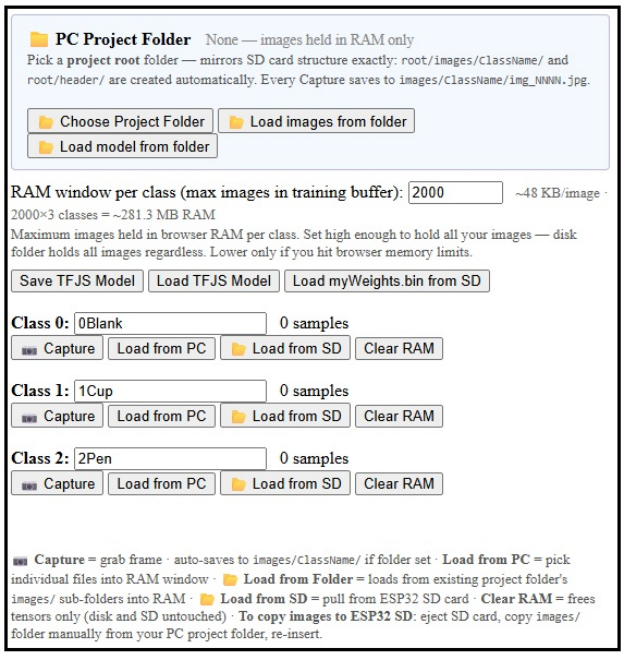

Figure 7: Example training images from the three-class reference dataset (0Blank, 1Cup, 2Pen) at 64×64 resolution. Images are captured under consistent indoor lighting with a plain background. The deliberate simplicity of this dataset illustrates the TinyML principle: a clean, task-specific dataset outperforms a large general one for a fixed-condition deployment problem.

An *Update Confusion Matrix* button runs the full dataset through the current model and renders a colour-coded table (diagonal green, off-diagonal red), providing an immediate per-class accuracy breakdown during or after training.

Table 3: CNN architecture at `INPUT_SIZE` = 64, identical to the on-device firmware. Total parameters: 20,595. The small parameter count is a feature, not a limitation: it matches the memory and inference-speed constraints of the ESP32-S3 and is sufficient for clean, task-specific classification.

Layer	**Output shape**	**Notes**	**Parameters**
Input	64×64×3	RGB, normalized [0,1]	—
Conv1 (3×3, 4f)	62×62×4	Leaky ReLU (α=0.1)	112
MaxPool 2×2	31×31×4	—	—
Conv2 (3×3, 8f)	29×29×8	Leaky ReLU (α=0.1)	296
Flatten	6,728	—	—
Dense (3 classes)	3	Softmax	20,187
Total			20,595

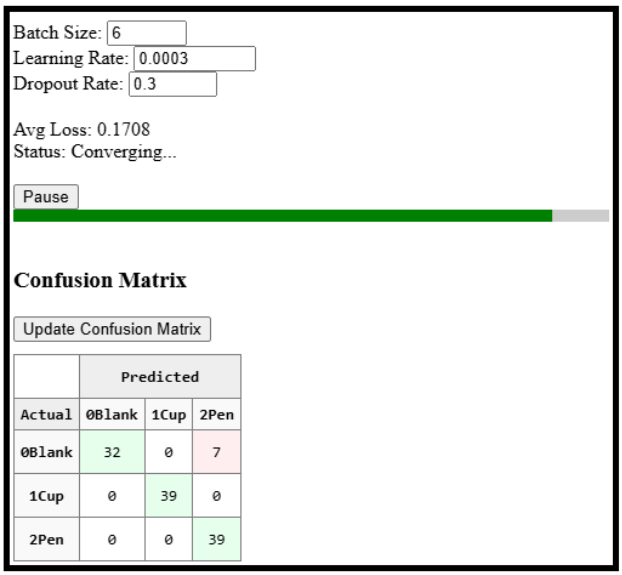

Figure 8: Confusion matrix from a three-class training run (0Blank, 1Cup, 2Pen) at 64×64 resolution after 20 epochs. Diagonal cells (green) indicate correct classifications; off-diagonal cells (red) identify class confusions. The confusion matrix is the primary diagnostic for deciding whether to collect more images for a specific class or adjust the training configuration.

4.5 Weight Export to SD Card

Section 5 exports browser-trained weights for deployment on the ESP32. The recommended deployment workflow is: (1) export weights and config to the PC project folder using the *Export myWeights.bin* and *Save config.json* buttons; (2) copy the `header/` subfolder from the PC project folder to the microSD card root; (3) insert the card and reboot the device. This single folder-copy step deploys both the trained model and any updated configuration simultaneously.

4.5.1 `myWeights.bin`

A compact binary file: an ASCII JSON header (architecture metadata and class labels, bounded by sentinel strings) followed by raw `float32` arrays in the transposed layout expected by the ESP32

firmware. The three weight tensors require layout transposition from TensorFlow.js internal format to the ESP32 C++ loop-order:

- Conv1: TF `[ky, kx, ic, f]` → ESP32 `[f, ky, kx, ic]`
- Conv2: TF `[ky, kx, ic, f]` → ESP32 `[f, ic, ky, kx]`
- Dense: TF `[flat, cls]` → ESP32 `[cls, f, y, x]`

Written to `/header/myWeights.bin` and loaded automatically at next boot via the three-tier priority system of Paper 1 [1]. The browser can also load this file from SD at any time, restoring on-device-trained weights into the live TensorFlow.js model for continued browser-side training or validation.

4.5.2 `myWeights.h`

A self-documenting C header containing all weight arrays as `float` initialisers with embedded instructions for enabling `#define USE_BAKED_WEIGHTS`. This format is identical to the header the ESP32 firmware itself generates after on-device training, making the two export paths interchangeable.

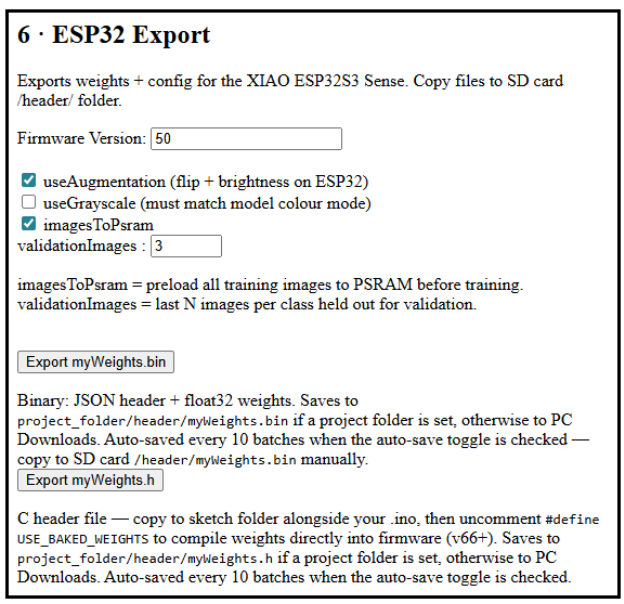

Figure 9: Section 5: Weight and configuration export. `myWeights.bin` can be downloaded to PC for manual transfer to the device SD card. The identical binary format is shared with the on-device firmware, making browser-trained weights directly deployable at the next device boot.

5 Conv2 Activation Heatmap

The live Conv2 activation heatmap is among the most pedagogically distinctive features of the system. In typical TinyML deployments the internal state of a running model is entirely opaque: the device produces a class label but exposes no information about which spatial regions of the input drove that prediction. The heatmap streaming protocol makes this visible in real-time.

When heatmap mode is enabled, the firmware serialises the maximum activation across all Conv2 filters at each spatial position into a compact byte array, base64-encodes it, and transmits it as:

```
HEATMAP:<rows>x<cols>:<base64-bytes>\n
```

Listing 2: Heatmap serial protocol (firmware to browser). One frame is sent per inference cycle while heatmap mode is active.

The browser decodes the payload, maps each byte through a five-segment blue → cyan → green → yellow → red colour ramp, and renders it on a scaled `<canvas>` element. At the default 64×64 input, the Conv2 output is 29×29, displayed at 6× scale (174×174 px).

The heatmap is also computable entirely in the browser from webcam input without ESP32 involvement, by extracting Conv2 activations from the TensorFlow.js model directly. This browser heatmap mode has no impact on ESP32 FPS.

The heatmap serves as a "glass box" diagnostic: a practitioner can hold their target object in front of the camera and observe whether the activation cluster follows the object or remains anchored to a background artefact—a bright surface, a consistent light gradient, or an irrelevant texture. If the heatmap shows high activation on the background rather than the object, the diagnosis is immediate: the dataset lacks background variation. More images with different backgrounds, or images taken at different positions, will correct it. To our knowledge, this is one of the first real-time activation visualizations deployed on a microcontroller-class device and streamed to a browser for interactive inspection. This feedback loop is unavailable in other low-cost TinyML deployments we are aware of.

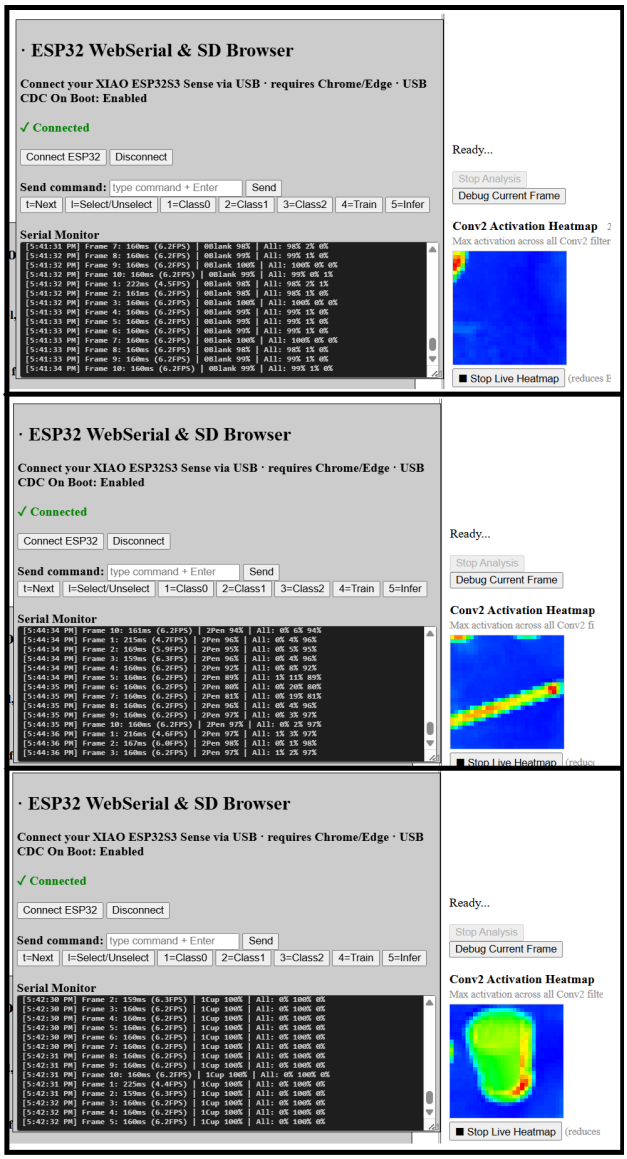

Figure 10: Live Conv2 activation heatmap streamed from the ESP32 during inference. Red = high activation; blue = low activation. The heatmap reveals which spatial regions drive the prediction. Here, activation is concentrated on the object of interest rather than the background, indicating that the model has learned a spatially appropriate feature rather than a spurious correlation. If activation instead clusters on the background, the practitioner knows immediately that more background-varied images are needed.

6 Experimental Evaluation

6.1 Setup

All measurements used the Seeed Studio XIAO ESP32-S3 Sense board mounted on the XIAO ML Kit expansion board (adding the 72×40 OLED display and IMU) with the three-class reference problem (0Blank, 1Cup, 2Pen) at 64×64 resolution under consistent indoor lighting. Browser training ran in Chrome 124 on a laptop with integrated GPU (WebGL acceleration enabled). On-device results are taken from Paper 1 [1]. Training configuration: batch size 6, learning rate 0.0003, dropout 0.3, **100 epochs** per browser run. All raw results, per-run confusion matrices, real-world inference images, and heatmap captures are available in the repository at the release tag noted on the title page.

6.2 Consistency Evaluation: Five Independent Runs

To assess training stability across dataset variation, five training runs were conducted with independently collected image sets (fresh captures per run, ∼30 images per class). Results are summarised in Table 4 and Figures 11–14.

Validation set note (v1.0.0): In the initial experimental design, the validation split was fixed at 3 images per class regardless of training set size. As the training set grew from the original 9 images/class to ∼30 images/class and beyond, the validation set remained at 3 images/class, producing an increasingly unbalanced train-to-validation ratio. A future version of these experiments will use a proportional validation split (e.g., 20% held-out per class) to provide more statistically stable accuracy estimates as dataset size increases.

Table 4: Five-run consistency evaluation on the three-class reference problem (0Blank, 1Cup, 2Pen). Each run used a freshly collected dataset of ∼30 images per class, 100 epochs, browser training (TensorFlow.js, WebGL). Percentages are reported for cross-run comparability; raw counts are available in the repository. Confusion matrices are computed on the held-out validation split.

Metric	**Mean**	**Std**	**Notes**
Final training loss	0.127	0.118	See Fig. 11
Final validation accuracy	95.6%	9.9%	See Fig. 12
Convergence epoch	10.2	6.5	See Fig. 13
Real-world inference Blank	87%	14%	See Fig. 15
Real-world inference cup	98.0%	2.1%	See Fig. 15
Real-world inference pen	83.0%	0.7%	See Fig. 15

6.3 Training Accuracy vs. Real-World Inference

A consistent finding across all five runs is that training and validation confusion matrices—which are computed on images captured under the same conditions as the training set—overstate real-world inference accuracy. When the model is evaluated on live camera images during deployment, accuracy can be lower, particularly for classes whose features are sensitive to small changes in distance, angle, or ambient light level.

Figure 15 summarises this gap across runs.

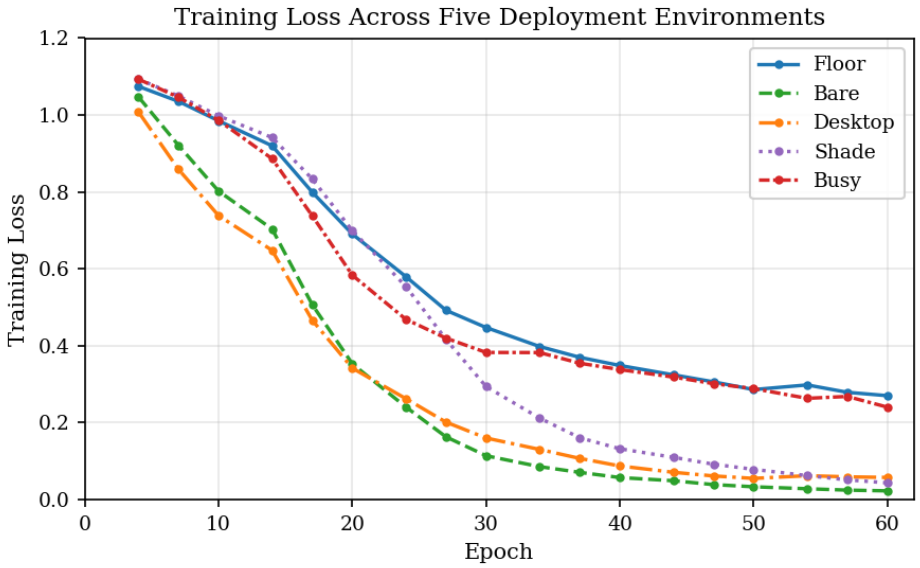

Figure 11: Training loss curves across five independent runs (100 epochs each, ∼30 images per class, fresh dataset per run). Consistent downward convergence across all runs indicates stable training dynamics despite dataset variability.

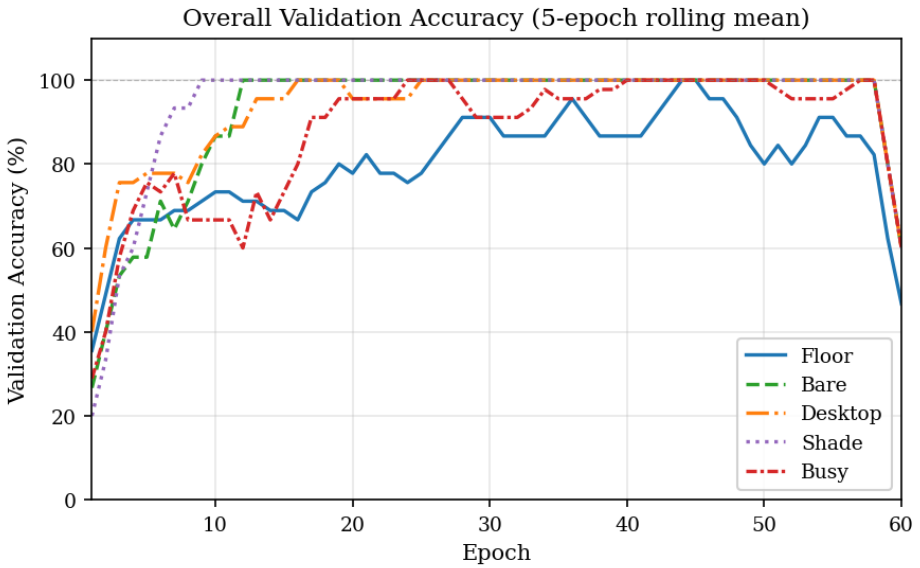

Figure 12: Validation accuracy across five runs. The validation split is held out at the start of each run and not used during training. Values are reported as percentages for comparability across runs with differing image counts.

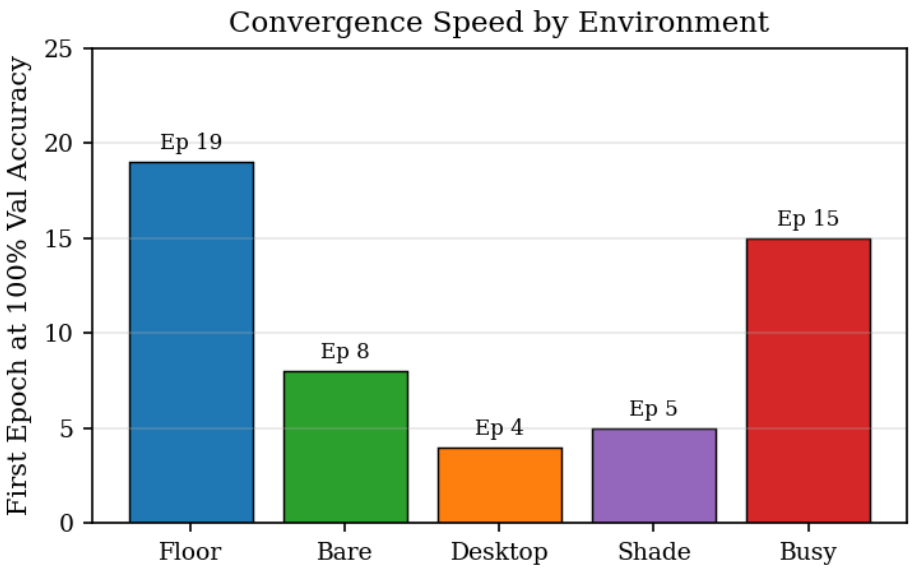

Figure 13: Epoch at which each run first reaches a stable convergence threshold. Early convergence in most runs reflects the task-specific nature of the dataset: once the model has seen sufficient examples of each class under fixed conditions, additional epochs yield diminishing returns.

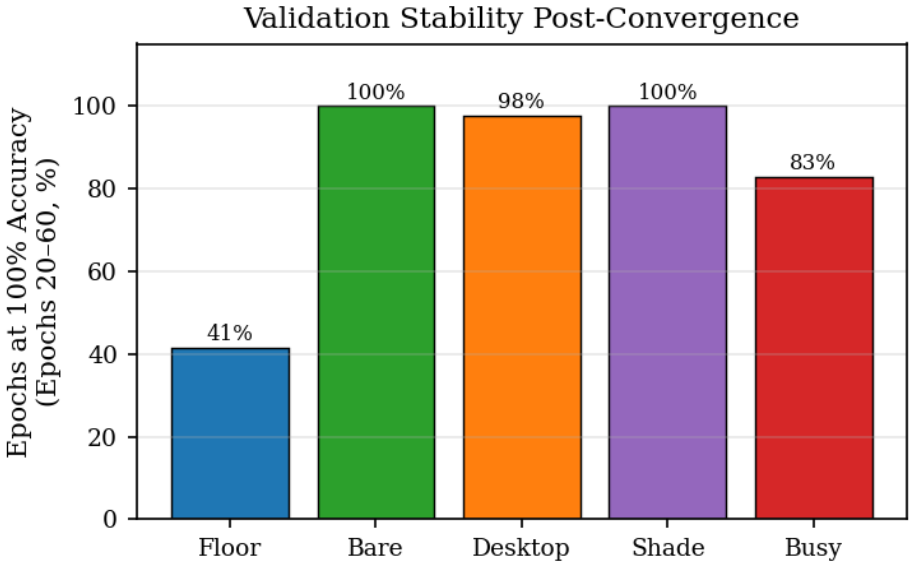

Figure 14: Validation accuracy stability (variance across the last 10 epochs of each run) across five test environments: *Floor* (objects placed on the floor), *Bare* (plain white paper under good lighting), *Desktop* (standard desk surface), *Shade* (very dark ambient lighting), and *Busy* (a paint-splattered workbench). Low variance indicates that training has reached a stable plateau; high variance suggests the model has not fully converged or the dataset is too small for the selected epoch count.

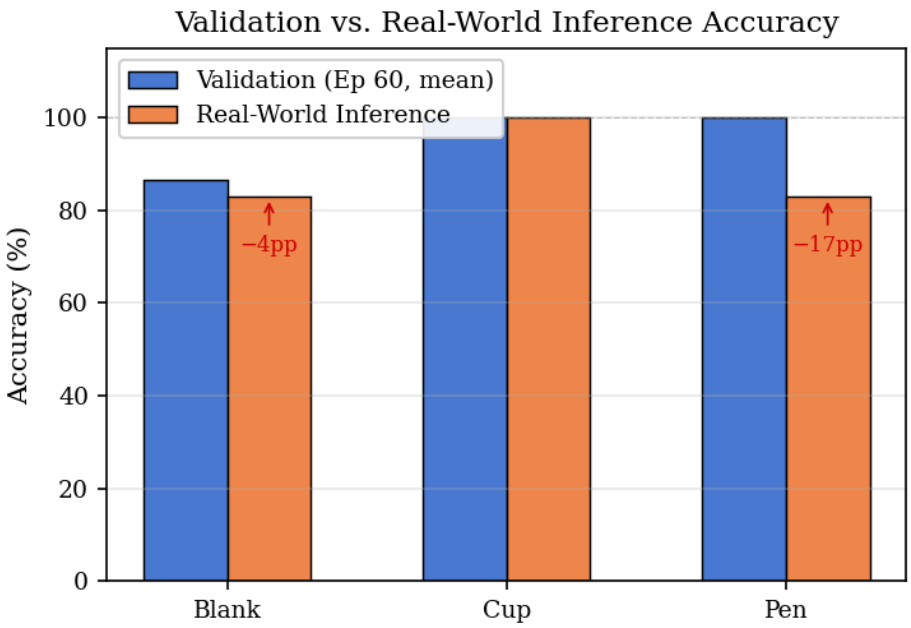

Figure 15: Validation accuracy (computed on held-out training-set images) versus real-world inference accuracy (computed on live camera images at deployment time) across five runs. The gap reflects the practitioner's own deployment conditions, not a flaw in the training pipeline. Closing this gap is achieved by capturing training images that more faithfully represent the range of positions and lighting variations the device will encounter.

This gap is both expected and informative. The confusion matrix is the correct tool for diagnosing *which classes* are confused during training; the real-world inference images (with live heatmaps, see Figure 16) are the correct tool for diagnosing whether the model has learned the object or a background artefact. A high confusion-matrix score with low real-world accuracy is a signal to collect images under more varied conditions—different distances, angles, and lighting levels.

An important secondary finding is that the models can be trained effectively under low light conditions. Provided the training images are captured under the same low-light conditions as deployment, the model generalises to that environment reliably. This is consistent with the deployment philosophy of Section 6.5: dataset representativeness of the actual deployment conditions matters more than absolute image quality.

6.4 Training Speed and Accuracy

Table 5 presents the key performance comparison. Browser-side TensorFlow.js training reaches the same approximate loss as a full on-device run (∼9 minutes) in approximately 1 minute for a representative dataset of ∼30 images per class—a 9× reduction in per-run iteration time. A complete collect–train–deploy cycle (image capture, browser training, weight export to PC folder, microSD copy, device reboot) is achievable in under 10 minutes, enabling several full iterations within a single class period. This speed difference stems from WebGL GPU parallelism versus the single-core floating-point unit of the ESP32-S3.

Browser training also typically reaches slightly higher accuracy than on-device training on the same dataset, because the WebGL backend provides a smoother optimization landscape than the ESP32

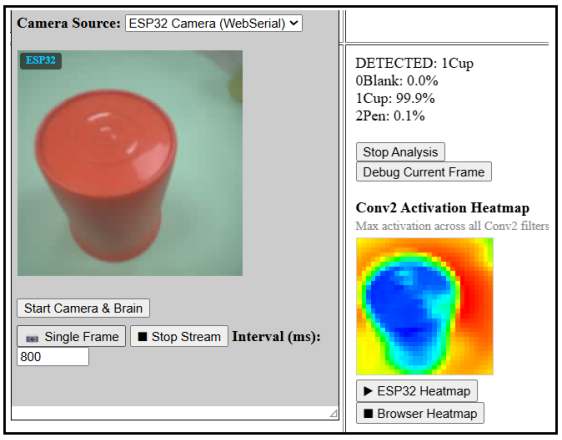

Figure 16: Live inference on the cup class under normal lighting: raw camera image (left), Conv2 activation heatmap (centre), and per-class confidence scores (right). The model correctly classified the cup with 99.9% confidence. The heatmap confirms that activation is concentrated on the cup object rather than the background, indicating that the model has learned a spatially appropriate feature. The cup class was the most consistently learnable across all five runs, likely because its circular rim and solid body produce a distinctive Conv2 response even at small image sizes.

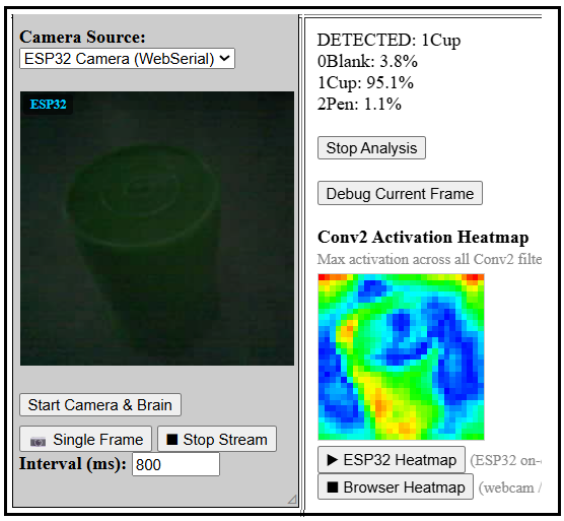

Figure 17: Live inference on the cup class under very low ambient light: raw camera image (left), Conv2 activation heatmap (centre), and per-class confidence scores (right). Despite the extremely dark image, the model correctly classified the cup with 95% confidence. The heatmap confirms that activation remains concentrated on the cup object rather than the background, demonstrating that a task-specific model trained under matching low-light conditions generalises reliably to that environment. This result illustrates the core TinyML deployment philosophy: dataset representativeness of the actual deployment conditions—including challenging lighting—matters more than absolute image brightness.

fixed-point FPU approximation. Weights exported from the browser and loaded onto the device perform at the browser-trained accuracy.

Table 5: Performance comparison: on-device (Paper 1) vs. browser-side training (this work). Reference dataset: 3 classes, ∼30 images/class, `INPUT_SIZE` = 64, 100 epochs. The 9× training speed improvement is the primary new result; the weight format and file size are identical between platforms.

Metric	**On-device (Paper 1)**	**Browser (this work)**	**Notes**
Training time to equiv. loss	∼9 min	∼1 min	9× faster
Full collect–train–deploy cycle	∼30 min	<3 min	incl. SD copy
Typical training accuracy	71–79%	75–88%	Browser has WebGL
Validation accuracy (3-class)	67–100%	67–100%	Same held-out split
Inference rate (device)	6.3 FPS	6.3 FPS	Runs on ESP32-S3
Heatmap FPS cost (device)	∼0.2 FPS	∼0.2 FPS	Serial overhead
Weight file (`.bin`)	∼83 KB	∼83 KB	Identical format
Firmware flash time	∼2 min (Arduino IDE)	∼30 s	Zero install

6.5 Dataset Size and Sufficiency

A recurring question in TinyML education is: how many images are enough? For a fixed-condition, specific-task classifier on this architecture, the answer is substantially smaller than general computer vision intuition suggests. Table 6 shows accuracy across dataset sizes for the three-class reference problem.

Table 6: Validation accuracy vs. dataset size per class for the 3-class reference problem (0Blank, 1Cup, 2Pen), browser training, 64×64 RGB, 100 epochs, consistent lighting and background.

Images/class	**Training accuracy**	**Validation accuracy**
9 (Dataset A)	65–72%	55–67%
∼20	70–80%	65–75%
38 (Dataset B)	75–88%	70–85%
100	80–90%	75–88%

The key insight is that these models are *intentionally* trained to overfit to their deployment conditions, which is desirable in fixed-condition TinyML applications. A model that achieves 100% training accuracy on its specific task under its specific conditions is behaving correctly; the relevant measure is real-world inference accuracy under those same conditions. Dataset *quality* and *representativeness of deployment conditions* matter more than quantity. A dataset of 60 images collected under the exact lighting and camera position of the deployment outperforms a dataset of 200 images collected under varied conditions that do not match deployment. This is the defining characteristic of TinyML data collection, and it separates TinyML from edge AI more broadly: edge AI seeks to bring general AI capability to local hardware; TinyML seeks to perform a specific task at minimum energy and cost. Both are valid goals, and both are served by this system.

7 WebSerial Communication Protocol

Table 7 summarises the full set of WebSerial commands exchanged between the browser and the firmware. All commands are newline-terminated ASCII strings sent at 115200 baud. The protocol uses explicit `_START`/`_END` sentinels for all multi-line responses so the browser parser can reassemble responses without relying on timing. An incomplete transaction (e.g., from a device reset) is detected by the absence of the closing sentinel and discarded.

Table 7: WebSerial command protocol between browser and ESP32 firmware.

Browser → ESP32	**ESP32 → Browser**	**Purpose**
`CAM_CAPTURE:WxH:Q`	`CAM_JPEG_START` → `CAM_JPEG:...` → `CAM_JPEG_END`	Request one JPEG frame
`CAM_STREAM_STOP`	—	Stop continuous stream
`SD_LIST:/path`	`SD_LIST_START` → `SD_FILE:...` → `SD_LIST_END`	Directory listing
`SD_READ:/path`	`SD_CONTENT_START` → `SD_LINE:...` → `SD_CONTENT_END`	Read text file
`SD_JPEG:/path`	`SD_JPEG_START` → `SD_JPEG:...` → `SD_JPEG_END`	Read JPEG from SD
`SD_JPEG_WRITE_START:/p:N` `SD_JPEG_CHUNK:b64` $\times N$ `SD_JPEG_WRITE_END`	`OK:JPEG_WRITE_DONE /path (NB)`	Write any binary or text file
`SD_DELETE:/path`	`OK:Deleted`	Delete file
`SD_RMDIR:/path`	`OK:Deleted`	Delete directory (recursive)
`HEATMAP_ON`	`HEATMAP:R×C:b64` per frame	Enable Conv2 heatmap stream
`HEATMAP_OFF`	—	Disable heatmap stream
`1`–`5`, `t`, `l`	(firmware menu state change)	Direct firmware menu navigation

8 Comparison to Paper 1 Workflow

Table 8 summarises the capabilities added by the browser companion relative to the standalone on-device workflow of Paper 1.

The two approaches are complementary. Paper 1 is the correct choice when the microcontroller must operate untethered, when raw data must never leave the device, or when understanding the C++ training loop at the instruction level is the pedagogical goal. Paper 2 is the correct choice when faster iteration, a richer management interface, or a lower barrier to first-session entry is the priority.

Table 8: Capability comparison: on-device (Paper 1) vs. WebSerial companion (Paper 2).

Capability	**Paper 1 (on-device)**	**Paper 2 (browser)**
Firmware installation	Arduino IDE required	Browser flash via `esptool-js` (Arduino IDE fallback for cold-offline start)
Data collection	On-device touch / SD	Webcam or ESP32 camera in browser
Training time	∼9 min/run (C++ FPU)	∼1 min/run (TF.js + WebGL); full cycle <10 min
SD file management	SD card ejection required	Full browser SD browser
Config adjustment	Recompile required	Live sync, zero recompile
Weight export	SD card only	SD direct or PC download
Weight import	SD boot auto-load	SD or PC upload
Confusion matrix	Serial text output	Colour-coded HTML table
Conv2 heatmap	Not available	Live streaming from firmware
Energy measurement	External PPK2 profiler	Not applicable (laptop training)
Raw data privacy	Never leaves device	Stays on local machine
LLM adaptability	Full C++ source	Single HTML file

9 Limitations

- **Browser compatibility.** WebSerial is supported only in Chromium-based browsers (Chrome 89+, Edge 89+). Firefox and Safari are not supported. The page runs fully locally by opening `index.html` directly in Chrome or Edge — no local HTTP server is required. The portable folder in the repository vendors all CDN libraries locally, so the complete page (excluding firmware flash) operates offline once that folder has been downloaded. WebSerial itself always requires a Chromium engine regardless of how the page is served.

- **SD card deployment via PC folder copy (recommended); browser-side SD write not yet active.** The primary recommended workflow is to save all trained artefacts—images, weights (`myWeights.bin`, `myWeights.h`), and `config.json`—to a PC project folder whose directory layout mirrors the SD card root exactly. The entire folder is then copied to the microSD card in a single manual operation before inserting into the device. This avoids serial throughput constraints entirely. The *Export myWeights.bin* button writes directly to the PC folder. A planned browser feature would automate this PC-to-SD transfer without manual card handling, but this capability is not active in v1.0.0.

- **No persistent in-browser model storage.** TensorFlow.js weights are held in browser memory and lost on page refresh unless explicitly exported via the export buttons. The *Save TFJS Model* button provides a JSON + binary download for browser-portable saves; *Export myWeights.bin* is the recommended path for device deployment.

- **Architecture lock-in.** The CNN filter counts (Conv1 = 4, Conv2 = 8) are fixed to match

the on-device firmware. Changing filter counts requires coordinated modification of both the JavaScript export transpositions and the firmware `#define` constants. This is explicitly a supported use case for LLM-assisted modification: the transposition logic and relevant constants are documented here and in the source file.

- **Model capacity.** The 20,595-parameter architecture is sufficient for simple classification on clean, task-specific datasets but is not suitable for complex or background-varied scenes. For harder problems, the recommended path is better data—more varied images under deployment conditions—rather than a larger model, because model size is bounded by the ESP32's available PSRAM for on-device inference.
- **CDN dependency.** The three JavaScript libraries (TensorFlow.js, JSZip, esptool-js) are loaded from public CDNs when the page is first opened with internet access. Once loaded, they are cached by the browser and the page operates fully offline. For environments where the browser cannot be pre-loaded with internet, the portable folder in the repository includes all three libraries vendored locally alongside `index.html`, requiring no CDN access at any point, except for firmware flash.

10 Repository and Reproducibility

The application is maintained at:

https://github.com/webmcu-ai/webmcu-vision-web

This paper corresponds to release tag `v1.0.0`:

https://github.com/webmcu-ai/webmcu-vision-web/releases/tag/v1.0.0

The release includes `index.html`, firmware.ino, companion firmware binaries (`.ino.bin`, `.ino.merged.bin`), all five-run evaluation datasets, per-run confusion matrices, real-world inference images with heatmaps, portable folder structure, and the figure source data. No build step is required: open `index.html` in Chrome or Edge and connect the ESP32 via USB.

The on-device firmware (Paper 1) is at:

https://github.com/webmcu-ai/on-device-vision-ai

Reproducibility checklist:

- Hardware: Seeed Studio XIAO ESP32-S3 Sense (8 MB PSRAM variant) with XIAO ML Kit expansion board (MicroSD slot, OLED display).
- Firmware: `webmcu-vision-web` v71 (`firmware.ino`), compiled with Arduino IDE 2.x, board package *esp32* v3.x, `U8g2` v2.x.
- Browser application: `webmcu-vision-web` `index.html` (release `v1.0.0`); TensorFlow.js 4.22.0, esptool-js 0.5.7, JSZip 3.10.1.
- Browser: Chrome 124+ or Edge 124+.
- Reference dataset: ∼30 images/class per run; 3 classes (0Blank, 1Cup, 2Pen); 5 independent runs with freshly collected images; consistent indoor lighting.
- Training configuration: `INPUT_SIZE` = 64, batch 6, lr 0.0003, 100 epochs, dropout 0.3.

- Deployment: save artefacts to PC project folder via *Export myWeights.bin* and *Save config.json*; copy `header/` folder to microSD card; reinsert and reboot device.
- All evaluation artefacts (per-run datasets, confusion matrices, inference images, heatmaps, figure source data) available at the `v1.0.0` release tag.

Both repositories are released under the MIT License.

11 Future Work

Paper 3 of this series will extend the framework to on-device audio classification using the XIAO ML Kit's built-in I2S microphone, with a companion browser application providing spectrogram visualization and audio data collection. Paper 4 will extend to IMU-based gesture and activity recognition using the ML Kit's on-board IMU. Both will follow the same template structure and be documented for LLM-assisted adaptation.

Planned additions to `webmcu-vision-web` include:

- **Proportional validation split.** Future experimental runs will use a fixed percentage (e.g., 20%) of each class as the held-out validation set, rather than the fixed 3-image split used in v1.0.0. This ensures that validation accuracy estimates remain statistically meaningful as training datasets scale beyond ∼30 images per class.
- **Offline-first firmware flashing.** In-browser firmware flashing via `esptool-js` works correctly when the page has been loaded with internet access. The portable folder already addresses the CDN dependency for offline use. A future improvement will ensure the flash workflow is clearly documented and tested for first-boot offline scenarios, and will explore caching strategies so that the flash library is retained across browser sessions without relying on the portable folder.
- **Manual PC-to-SD card transfer.** The current workflow requires manually copying the PC project folder to the microSD card. A planned browser feature will automate this transfer step, eliminating the need to eject and reinsert the card between training and deployment, but this capability is not active in v1.0.0.
- Dynamic multi-class UI generation driven by `numClasses` read from `config.json`, removing the current three-class hard-coding.
- INT8 quantization export for compatibility with Espressif's ESP-NN production inference kernels [11], providing a natural follow-on exercise for students who have understood the full-precision baseline.
- WebGPU acceleration (replacing WebGL) for further training speed improvements as WebGPU browser support matures.
- Offline-capable bundle (all CDN libraries vendored locally) for use in restricted-network classrooms and field deployments in the global south.
- Percentage/count toggle in the confusion matrix display, aligning the in-browser diagnostic view with the percentage-normalized values used in published results.

LLM-adaptable design. The single-file architecture has a secondary benefit beyond zero-install deployment: it fits within the context window of current LLM assistants, making the entire codebase available for targeted modification in a single conversation. A practitioner can describe a new sensor modality, hardware variant, or classification task to an LLM and receive a targeted diff to the relevant section of the file. The fixed naming conventions, consistent comment structure, and self-contained module layout are all chosen to make these modifications deterministic and verifiable. This design principle extends to the paper series itself: each paper documents its companion repository precisely enough that an LLM-assisted practitioner can reproduce, adapt, and extend the system without deep software engineering expertise.

Community contributions are welcomed as GitHub issues and pull requests at https://github.com/webmcu-ai under the MIT License.

12 Conclusion

`webmcu-vision-web` is a complete, private, browser-based machine learning training and deployment system for TinyML vision on the Seeed Studio XIAO ESP32-S3 Sense as the XIAO ML kit. In a single HTML file it provides: zero-install firmware flashing; a live serial monitor; a full SD card browser with image preview and inline editing; zero-recompile hyperparameter adjustment via `config.json` live-sync; webcam and ESP32 OV2640 camera data collection for task-specific datasets; TensorFlow.js CNN training completing in approximately 1 minute for a representative three-class dataset of $\sim$30 images per class; a confusion matrix; weight export in binary and C header formats; and live Conv2 activation heatmap streaming from the running device.

Together with the concepts from the on-device Arduino firmware of Paper 1, the system covers the full embedded ML lifecycle: flash the device from the browser, collect images under your exact deployment conditions, train in the browser, save all artefacts to a PC project folder, copy the folder to the microSD card, and reboot the device to run inference at 6.3 FPS. This complete collect–train–deploy cycle takes under 10 minutes for a three-class problem, and can be repeated multiple times within a single class period or work session.

A central design principle is that TinyML works best when the dataset matches the deployment conditions exactly. These models are intentionally trained to overfit to their specific deployment environment—a specific object, under specific lighting, at a specific position. This is a feature of TinyML, not a limitation: real-world inference accuracy is the relevant measure, and it is maximised by capturing training images that faithfully represent the deployment context. The five-run consistency evaluation presented here demonstrates this principle in practice and provides reproducible baselines at the v1.0.0 release tag.

The repository is structured as a living template for LLM-assisted adaptation, enabling educators, small businesses, and researchers in any environment to modify the system for their specific hardware, sensors, and classification tasks. No data leaves the local machine at any stage. All source is MIT-licensed at https://github.com/webmcu-ai/webmcu-vision-web.

Acknowledgements

The author thanks Brian Plancher (Dartmouth College) for his generous support and suggestions during this project; his contributions to the TinyML4D community [8] and to Paper 1 of this series

provided both technical grounding and useful context for the browser-based companion described here.

The author used free tier LLM assistants—Claude (Anthropic), ChatGPT (OpenAI), and Gemini (Google)—for structural editing, code review and research, and section-level refactoring of the single-file application; all technical decisions, experimental results, and architectural choices are the author's own. The LLM-assisted workflow described in this paper is itself a demonstration of the approach: a practitioner with domain knowledge of the hardware and the problem can use LLM tools to accelerate development of a working embedded ML system without requiring deep software engineering expertise.

References

[1] J. Ellis, "On-Device Vision Training, Deployment, and Inference on a Thumb-Sized Microcontroller," arXiv preprint, April 2026. https://github.com/webmcu-ai/on-device-vision-ai

[2] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *Proc. Int. Conf. Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1412.6980

[3] Edge Impulse Inc., "Edge Impulse: Machine Learning for Embedded Systems," 2024. https://edgeimpulse.com

[4] Google, "Teachable Machine," 2024. https://teachablemachine.withgoogle.com

[5] Arduino, "Arduino IDE," 2024. https://www.arduino.cc/en/software

[6] TensorFlow Authors, "TensorFlow Lite for Microcontrollers," 2024. https://www.tensorflow.org/lite/microcontrollers

[7] W3C Web Incubator Community Group, "Web Serial API," 2024. https://wicg.github.io/serial/

[8] B. Plancher, S. Büttrich, J. Ellis, et al., "TinyML4D: Scaling Embedded Machine Learning Education in the Developing World," *Proc. AAAI Symposium Series*, pp. 508–515, 2024. https://doi.org/10.1609/aaaiss.v3i1.31265

[9] M. Smilkov et al., "TensorFlow.js: Machine Learning for the Web and Beyond," arXiv preprint arXiv:1901.05350, 2019. https://arxiv.org/abs/1901.05350

[10] Espressif Systems, "esptool-js: JavaScript implementation of esptool," GitHub, 2024. https://github.com/espressif/esptool-js

[11] Espressif Systems, "ESP-DL: Espressif Deep Learning Library," GitHub, 2024. https://github.com/espressif/esp-dl